



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Enterprise ICT Architectures (Module 1) - Third Project De- livery MongoDB

Author(s): **Alberto Lenoci - 11095788**

Elena Casaleggi - 10706547

Matteo Lo Giudice - 10829241

Veronica Fatigati - 11095787

Group Number: **4**

Academic Year: 2024-2025

Contents

Contents	i
1 Introduction	1
2 MongoDB Representation	3
3 Queries	5
List of Figures	29

1 | Introduction

"The 3rd Project Delivery is built starting from the 1st Project Delivery. In particular, we ask you to choose a set of minimum 5 entities from your ER Model and develop a MongoDB Database. Define the basic structure of each document (i.e., name, type, etc.) and the containment hierarchy (i.e., if a document representing an entity has an array of sub-documents representing an entity, describe the main document accordingly). You can re-use the data you created for the first delivery, although you'll have to reshape it to be coherent with the new MongoDB structure. Define at least 10 documents for each entity. If an entity's document is contained in another entity, please define at least 20 of them in total (i.e., across all documents, e.g., 2 documents for each of the 10 main documents). Leave no empty arrays."

For the third delivery of this EICTA group project, we migrated the SQL database of the *WellDressed fashion company* to the schema-less document-oriented database MongoDB.

The **schema-less nature** of MongoDB is one of its defining characteristics, allowing us to dispense with a fixed schema defined in advance. This flexibility means that the structure of the database can be adapted dynamically as new requirements emerge, without necessitating costly schema migrations or disruptions to existing data. Such an approach is particularly advantageous for projects where requirements evolve over time or where the data itself is inherently diverse.

MongoDB's design philosophy is also highly **coherent with object-oriented programming principles**. Data is stored in collections of documents, where each document corresponds to an object-like representation of an entity. These documents are stored physically as JSON-like objects (technically BSON, a binary-encoded JSON format), which are human-readable when exported to text files, offering a significant advantage over relational models. In relational databases, entities and relationships are stored in tabular formats that require specialized tools to interpret, while graph databases use proprietary formats for storing nodes and edges, often obscuring direct access.

The core structure of a MongoDB database consists of **collections**, which serve as logical

groupings of **documents**, analogous to tables in relational databases. Each document is self-contained and flexible, as there is no requirement to predefine data types for attributes. This lack of rigidity allows for seamless handling of heterogeneous data, making MongoDB an ideal choice for use cases where entities may have varying attributes or where attributes themselves may evolve over time.

Every document in MongoDB is uniquely identified by a mandatory identifier, typically stored in the `_id` field. This ensures that documents are distinguishable within a collection, functioning similarly to a primary key in relational databases. The unique identifier is central to maintaining data integrity and facilitating efficient retrieval and updates of specific documents.

A powerful feature of MongoDB is its support for embedded documents, or sub-documents. This hierarchical structuring capability enables us to encapsulate related data within a single document, reducing the need for complex joins typical in relational databases.

The flexibility offered by MongoDB necessitates thoughtful design decisions regarding document structure. Unlike relational databases, where normalization principles dictate the schema, MongoDB requires developers to tailor document models to the specific needs of the application and its users. Factors such as query patterns, data access frequency, and expected scalability must guide these decisions to ensure optimal performance and maintainability.

To perform the migration, we utilized **MongoDB Relational Migrator**, a tool that allowed us to establish a seamless connection between our existing MySQL database and the **MongoDB Atlas Cluster**. This ensured that every entity, attribute, and relationship from the SQL database was preserved without compromising on size, accuracy, consistency, or cardinalities. The resulting MongoDB schema was not only faithful to the original relational model but also leveraged the advantages of schema-less design to offer greater flexibility and scalability.

2 | MongoDB Representation

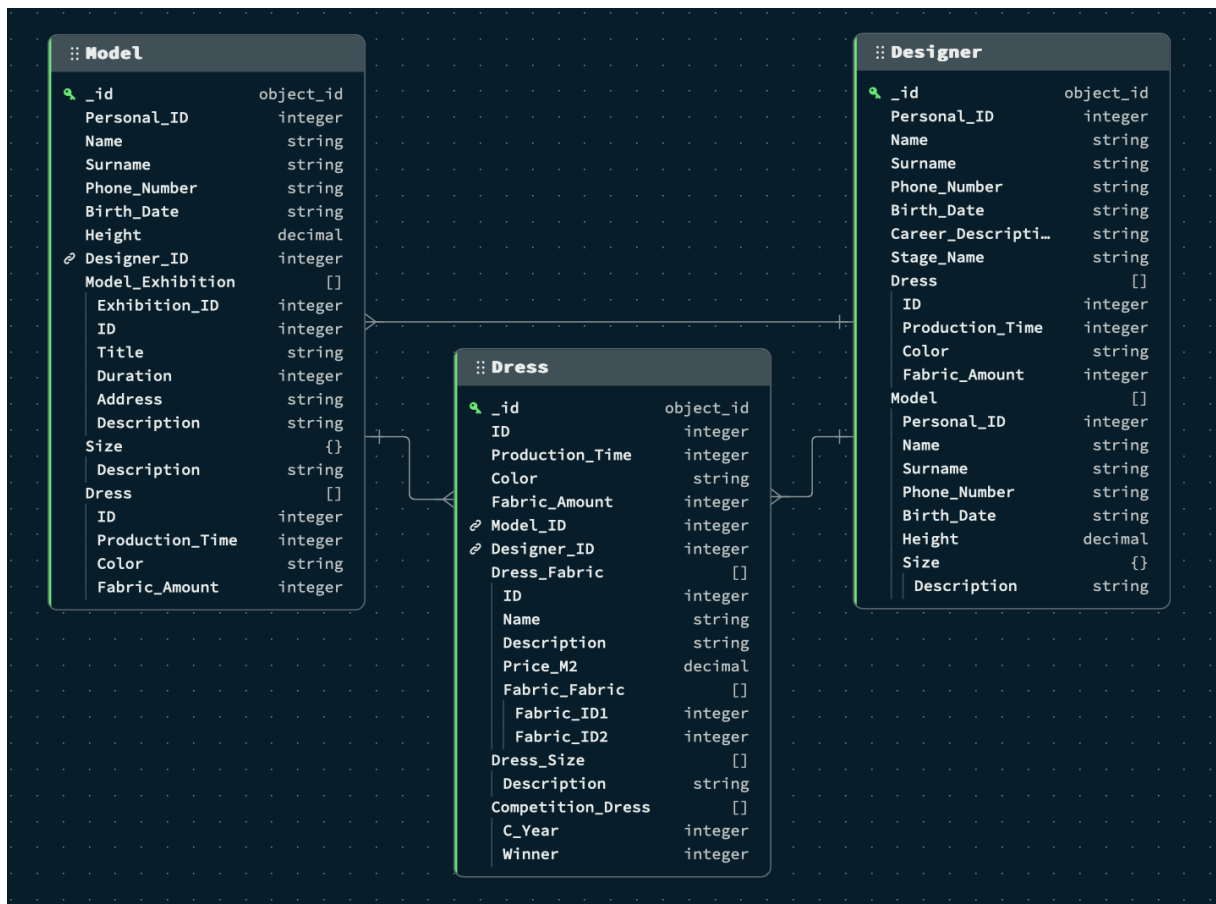


Figure 2.1: MongoDB Collections Structure

The migration tool allowed us to customize the structure of our documental database, which we have altered in relation to the SQL schema in order to allow for more efficient queries. Doing so, we have taken the following steps:

- Created collections of the Dress, Model and Designer entities.
- Other entities have not been made into collections, but integrated in the other 3 as sub-documents
- Bridge tables have also been eliminated in favor of replacing them with sub-documents

and sub-document arrays.

In particular:

- Fabric, Fabric_Fabric, Dress_Size, Size, Competition_Dress and Competition have been nested inside the Dress collection
- Exhibition, Model_Exhibition, Size and Dress have been nested inside the Model collection
- Dress, Model and Size have been nested inside the Designer collection

Furthermore, we have removed the showcasing of redundant IDs from the sub-documents, displaying only relevant information and increasing the efficiency of database querying.

3 | Queries

5 creation/update/delete queries (at least 1 for each type)

Query 1

Create a new dress with random data.

```
db.Dress.insertOne({
  "ID": 26,
  "Production_Time": 3,
  "Color": "Orange",
  "Fabric_Amount": 6,
  "Model_ID": 23,
  "Designer_ID": 4,
  "Competition_Dress": [
    {"C_Year": 2010,
     "Winner": 0}
  ],
  "Dress_Fabric": [
    {"Description": "Soft",
     "ID": 1,
     "Name": "Cotton",
     "Price_M2": {"$numberDecimal": "62.01"}}
  ],
  "Dress_Size": [{"Description": "Large"}]
})
```

```
> db.Dress.insertOne({
  "ID": 26,
  "Production_Time": 3,
  "Color": "Orange",
  "Fabric_Amount": 6,
  "Model_ID": 23,
  "Designer_ID": 4,
  "Competition_Dress": [
    {
      "C_Year": 2010,
      "Winner": 0
    }
  ],
  "Dress_Fabric": [
    {
      "Description": "Soft",
      "ID": 1,
      "Name": "Cotton",
      "Price_M2": {
        "$numberDecimal": "62.01"
      }
    }
  ],
  "Dress_Size": [
    {
      "Description": "Large"
    }
  ]
})
< {
  acknowledged: true,
  insertedId: ObjectId('6749d947578e17c290222ec0')
}
```

Figure 3.1: Query 1 Mongoshell Output

This query uses the function *insertOne()* to add a new dress to the *Dress* collection. The document inserted has an *ID* equal to 26 and contains random data for all other at-

tributes, including production time, color, fabric amount, associated model and designer, competition details, fabric specifications, and size. The *insertOne()* function ensures that the new document is added without impacting existing documents in the collection.

Query 2

Update the designer assigned to Model.Personal_ID (18) from Designer.Personal_ID (10) to Designer.Personal_ID (11).

```
db.Model.updateOne(  
  {"Personal_ID":18},  
  {"$set":{"Designer_ID":11}}  
)
```

```
> db.Model.updateOne(  
  {"Personal_ID":18},  
  {"$set":{"Designer_ID":11}}  
)  
< {  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 1,  
  modifiedCount: 1,  
  upsertedCount: 0  
}
```

Figure 3.2: Query 2 Mongoshell Output

The query uses the function *updateOne()* to update a single document in the *Model* collection. Specifically, it modifies the *Designer_ID* field for the first document that matches the filter criteria.

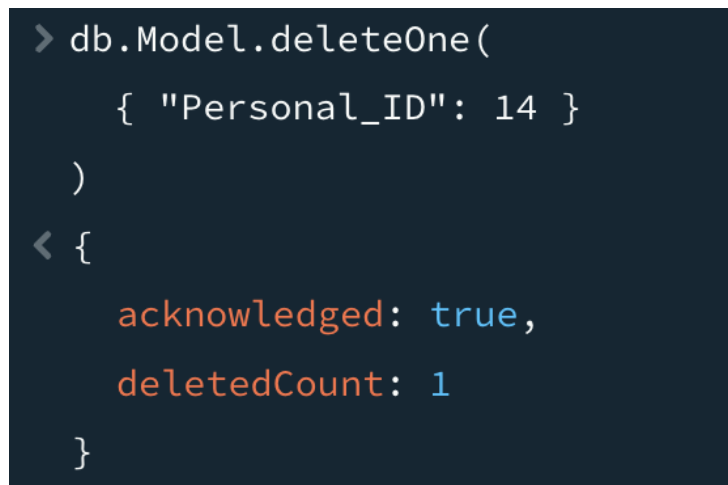
In this case, the filter *{"Personal_ID": 18}* identifies the document where the *Personal_ID* field has a value of 18. The *\$set* operator is then used to update the *Designer_ID* field, setting its value to 11.

The *updateOne()* function ensures that only the first document matching the filter is updated, even if multiple documents meet the criteria.

Query 3

Delete Model with *Personal_ID* equal to 14.

```
db.Model.deleteOne(  
  {"Personal_ID": 14 }  
)
```



```
> db.Model.deleteOne(  
  { "Personal_ID": 14 }  
)  
< {  
  acknowledged: true,  
  deletedCount: 1  
}
```

Figure 3.3: Query 3 Mongoshell output

This query uses the function *deleteOne()* to remove a single document from the *Model* collection. Specifically, it eliminates the document where the field *Personal_ID* has a value of 14.

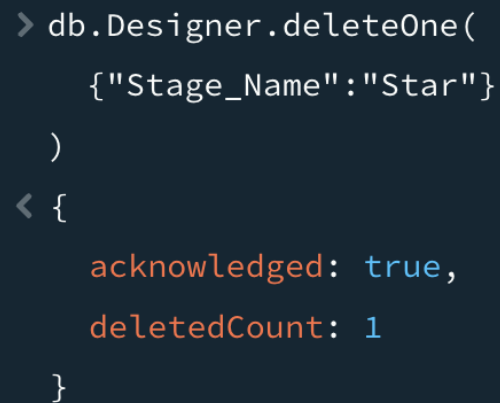
The filter *{"Personal_ID": 14}* identifies the document to be deleted. The *deleteOne()* function ensures that only the first matching document is removed, even if multiple documents meet the criteria.

The MongoShell output confirms the successful execution of this operation, with the document corresponding to *Personal_ID = 14* being permanently removed from the database.

Query 4

Delete a Designer whose Stage Name is “Star”.

```
db.Designer.deleteOne(  
  {"Stage_Name": "Star"}  
)
```

A screenshot of a terminal window with a dark background. It shows the execution of a MongoDB delete query. The prompt is a green greater-than sign. The command is `db.Designer.deleteOne({"Stage_Name": "Star"})`. The output is a green less-than sign followed by a JSON object: `{ "acknowledged": true, "deletedCount": 1 }`.

```
> db.Designer.deleteOne(  
  {"Stage_Name": "Star"}  
)  
< {  
  acknowledged: true,  
  deletedCount: 1  
}
```

Figure 3.4: Query 4 Mongoshell output

This query, similar to the previous one, uses the `deleteOne()` function to remove a single document from the *Designer* collection. The document to be deleted is identified by specifying the corresponding *Stage Name*.

Query 5

Update the size of a model from "Large" to "Small".

```
db.Model.updateOne(  
  {"Size": {"Description": "Large"}},  
  {$set: {"Size": {"Description": "Small"}}}  
)
```

```
> db.Model.updateOne(
  {"Size":{"Description":"Large"}},
  {$set:{"Size":{"Description":"Small"}}}
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Figure 3.5: Query 5 Mongoshell output

This query uses the *updateOne()* function along with the *\$set* operator to locate a model with the size *Large*. Once a matching document is found, the corresponding size is updated from *Large* to *Small*.

2 queries with projections and logical query operators

Query 1

Find the models born before 1998 **AND** taller than 180 cm, return model's name, surname, birthdate (as temporary field), height.

```
db.Model.find(
  {"$and": [
    {"Birth_Date":{"$lte": new Date("1998-01-01")}},
    {"Height":{"$gte":180.00}}]},
  {"Name": 1, "Surname": 1, "Height": 1, "year": { "$year": "
    $Birth_Date" }}
)
```

```
> db.Model.find(
  {"$and": [
    {"Birth_Date":{"$lte": new Date("1998-01-01")}},
    {"Height":{"$gte":180.00}}]},
  {"Name": 1, "Surname": 1, "Height": 1, "year": { "$year": "$Birth_Date" }}
)
< {
  _id: ObjectId('6749c93b0e659675aa2abdaa'),
  Name: 'Erick',
  Surname: 'Tapp',
  Height: Decimal128('183.05'),
  year: 1991
}
{
  _id: ObjectId('6749c93b0e659675aa2abdac'),
  Name: 'Sile',
  Surname: 'McCritichie',
  Height: Decimal128('183.30'),
  year: 1996
}
{
  _id: ObjectId('6749c93b0e659675aa2abdb9'),
  Name: 'Vanny',
  Surname: 'Tweedell',
  Height: Decimal128('183.09'),
  year: 1990
}
{
  _id: ObjectId('6749c93b0e659675aa2abdc2'),
  Name: 'Gasparo',
  Surname: 'Thoroughgood',
  Height: Decimal128('187.39'),
  year: 1992
}
```

Figure 3.6: Query 1 Mongoshell output

This query selects documents from the *Model* collection and uses the *\$and* logical operator

to identify models who meet two conditions: being born before 1998, by setting a condition on their *Birth_Date* using the *\$lte* operator, and being taller than 180 cm, by setting a condition on their *Height* using the *\$gte* operator. Since the *\$and* operator is employed, both conditions must be satisfied simultaneously.

Regarding projections, they are used here to return only specific attributes by assigning a value of *1* to the desired fields. Any field not explicitly listed is excluded from the result. Additionally, a temporary field is created using the *\$year* operator, which extracts and displays the year of birth of the model.

Query 2

Find dresses that won a competition **AND** are made either of wool **OR** cashmere, return Dress_ID and C_Year.

```
db.Dress.find(
  {"$and": [
    {"Competition_Dress.Winner": 1},
    {"$or": [
      {"Dress_Fabric.Name": "Wool"}, {"Dress_Fabric.Name": "Cashmere"}
    ]}],
  {"ID": 1, "Competition_Dress.C_Year": 1}
)
```



```
> db.Dress.find(
  {"$and": [
    {"Competition_Dress.Winner": 1},
    {"$or": [
      {"Dress_Fabric.Name":"Wool"}, {"Dress_Fabric.Name":"Cashmere"}]
    ]},
  {"ID": 1, "Competition_Dress.C_Year": 1}
)
< {
  _id: ObjectId('6749c93b0e659675aa2abdc8'),
  ID: 6,
  Competition_Dress: [
    {
      C_Year: 2014
    }
  ]
}
{
  _id: ObjectId('6749c93b0e659675aa2abdce'),
  ID: 12,
  Competition_Dress: [
    {
      C_Year: 1998
    }
  ]
}
{
  _id: ObjectId('6749c93b0e659675aa2abdd8'),
  ID: 22,
  Competition_Dress: [
    {
      C_Year: 2001
    }
  ]
}
```

Figure 3.7: Query 2 Mongoshell output

This query utilizes the logical operators *\$and* and *\$or* to select documents from the *Dress* collection. It returns dresses that have won a competition and are made either of wool or

cashmere, by setting conditions on *Competition_Dress* and *Dress_Fabric*.

The *\$or* operator specifies that a dress can be made of either wool or cashmere to meet the condition, while the *\$and* operator ensures that a dress satisfying the fabric condition must also have won a competition.

Projections are used to limit the returned fields to only the *ID* of the dress and the year of the competition, excluding all other attributes from the result.

2 queries with limit, sort, and comparison query operators

Query 1

Find the tallest models whose *Personal_ID* is between 15 and 20.

```
db.Model.find(  
  {"Personal_ID":{"$in":[15,16,17,18,19,20]}},  
  {"Name":1,"Surname":1,"Height":1}  
) .sort({"Height":-1})  
) .limit(3)
```

```
> db.Model.find(
  {"Personal_ID":{"$in":[15,16,17,18,19,20]}},
  {"Name":1,"Surname":1,"Height":1}
).sort({"Height":-1}
).limit(3)

< {
  _id: ObjectId('6749c93b0e659675aa2abdb9'),
  Name: 'Vanny',
  Surname: 'Tweedell',
  Height: Decimal128('183.09')
}
{
  _id: ObjectId('6749c93b0e659675aa2abdb8'),
  Name: 'Waylin',
  Surname: 'Mackilpatrick',
  Height: Decimal128('179.11')
}
{
  _id: ObjectId('6749c93b0e659675aa2abdba'),
  Name: 'Rogers',
  Surname: 'Cowsby',
  Height: Decimal128('169.21')
}
```

Figure 3.8: Query 1 Mongoshell output

This query targets the *Model* collection and identifies models whose *Personal_ID* falls within a specified range, using the *\$in* operator. The *sort()* function is applied to the *Height* attribute to arrange the models in descending order, from the tallest to the shortest.

Projections are used to specify the attributes to be returned, while the *limit()* function restricts the output to the three tallest models within the specified range.

Query 2

Find the top 3 oldest designers born from 1995 onwards.

```
db.Designer.find(
  {"Birth_Date": {"$gte": new Date("1995-01-01")}},
  {"Stage_Name": 1, "Birth_Date": 1}
).sort({"Birth_Date": 1}
).limit(3)
```

```
> db.Designer.find(
  {"Birth_Date": {"$gte": new Date("1995-01-01")}},
  {"Stage_Name": 1,"Birth_Date":1}
).sort({"Birth_Date":1}
).limit(3)
< {
  _id: ObjectId('6749c93b0e659675aa2abd9b'),
  Birth_Date: 1996-08-30T22:00:00.000Z,
  Stage_Name: 'Amy'
}
{
  _id: ObjectId('6749c93b0e659675aa2abd95'),
  Birth_Date: 1996-10-01T22:00:00.000Z,
  Stage_Name: 'Aldo'
}
{
  _id: ObjectId('6749c93b0e659675aa2abd97'),
  Birth_Date: 1997-07-14T22:00:00.000Z,
  Stage_Name: 'Echo'
}
```

Figure 3.9: Query 2 Mongoshell output

This query retrieves documents from the *Designer* collection by applying specific filter conditions. The filter targets the *Birth_Date* field, using the *\$gte* operator to set a minimum threshold.

Projections are employed to return only selected attributes, specifically *Stage_Name* and *Birth_Date*. The *sort()* function is then applied to order the results in ascending order, from the youngest to the oldest designer. Finally, the *limit()* function restricts the query output to the first three entries.

1 query element with query operators

Query 1

Find dresses that won a competition and check if the *C_Year* attribute is an integer.

```
db.Dress.find({"Competition_Dress.Winner": 1, "Competition_Dress.C_Year": {"$type": "int"}})
```

```
> db.Dress.find(
  {"Competition_Dress.Winner": 1,
  "Competition_Dress.C_Year": {"$type": "int"}}
).limit(1)
< {
  _id: ObjectId('6749c93b0e659675aa2abdc4'),
  ID: 2,
  Production_Time: 4,
  Color: 'Turquoise',
  Fabric_Amount: 5,
  Model_ID: 2,
  Designer_ID: 2,
  Competition_Dress: [
    {
      C_Year: 2010,
      Winner: 1
    }
  ],
  Dress_Fabric: [
    {
      Description: 'Soft',
      ID: 1,
      Name: 'Cotton',
      Price_M2: Decimal128('63.5602500000000000'),
      Fabric_Fabric: [
        {
          Fabric_ID1: 1,
          Fabric_ID2: 14
        }
      ]
    }
  ],
  Dress_Size: [
    {
      Description: 'X-Large'
    }
  ]
}
```

Figure 3.10: Query 1 Mongoshell output

This query targets the *Dress* collection and selects dresses that meet specific conditions by filtering on the *Competition_Dress* field. Specifically, it checks that *Competition_Dress.Winner* is set to *true*, indicating that the dress won a competition. Additionally, the query uses the *\$type* operator to ensure that the competition year is stored as an integer.

1 query with a simple aggregation

Query 1

Group dresses by color.

```
db.Dress.aggregate(  
  {$group: {_id: "$Color", count: {$sum: 1}}}   
)
```

```
> db.Dress.aggregate(  
  {"$group": {"_id": "$Color", count: {"$sum": 1}}}   
)  
< {  
  _id: 'Orange',  
  count: 5  
}  
{  
  _id: 'Crimson',  
  count: 1  
}  
{  
  _id: 'Purple',  
  count: 1  
}  
{  
  _id: 'Goldenrod',  
  count: 2  
}  
{  
  _id: 'Red',  
  count: 1  
}  
{
```

Figure 3.11: Query 1 Mongoshell output

This query uses the *aggregate()* function along with the *\$group* operator to group documents in the *Dress* collection based on their *Color* attribute. For each color, it calculates the total number of dresses by leveraging the *\$sum* operator, which increments a counter for every document in each group.

The resulting aggregation provides a summary of the number of dresses available for each color, enabling an easy analysis of color distribution within the collection.

1 query with an unwind and an aggregation

Query 1

Group models by Exhibition.

```
db.Model.aggregate(  
  [{ $unwind: "$Model_Exhibition" },  
  { $group: { _id: "$Model_Exhibition.Exhibition_ID",  
    models: { $push: { Name: "$Name", Surname: "$Surname", Height: "$Height",  
      Phone_Number: "$Phone_Number" } } } } ]  
)
```

```
> db.Model.aggregate(
  [{ $unwind: "$Model_Exhibition" },
  { $group: { _id: "$Model_Exhibition.Title",
    models: { $push: { Name: "$Name", Surname: "$Surname" } } } } ]
)
< {
  _id: 'Metropolis Muse',
  models: [
    {
      Name: 'Arvie',
      Surname: 'Leverett'
    }
  ]
}
{
  _id: 'Desert Bloom',
  models: [
    {
      Name: 'Ignacius',
      Surname: 'Goatman'
    }
  ]
}
{
  _id: 'Naïve Chic',
  models: [
    {
      Name: 'Erick',
      Surname: 'Tapp'
    },
    {
      Name: 'Leyla',
      Surname: 'Foakes'
    }
  ]
}
```

Figure 3.12: Query 1 Mongoshell output

This query is performed on the *Model* collection and utilizes the *aggregate()* function to process the data. The *\$unwind* operator is applied to the *Model_Exhibition* array, splitting it into multiple documents, each corresponding to a single participation in an exhibition. This transformation effectively normalizes the data, creating one document per model for each exhibition they participated in.

Subsequently, the documents are grouped by *Exhibition_ID* using the *\$group* operator. Within each group, the *\$push* operator is employed to construct an array of models associated with the same exhibition. This array includes detailed information for each model, such as their *Name*, *Surname*, *Height*, and *Phone_Number*.

1 query with an unwind, an aggregation, and followed by a filter

Query 1

Find all designers that designed dresses whose *Production_time* is lower than or equal to 3 and then group by color.

```
db.Designer.aggregate({$unwind: "$Dress"}, {$match: {"Dress.Production_Time": {$lte: 3}}}, {$group: {_id: "$Dress.Color", designers: {$addToSet: {"StageName": "$Stage_Name"}}}})
```

```

> db.Designer.aggregate(
  {$unwind: "$Dress"},
  {$match: {"Dress.Production_Time": {$lte: 3}}},
  {$group: {_id: "$Dress.Color", designers: {$addToSet: {"StageName": "$Stage_Name"}}}}
)
< {
  _id: 'Orange',
  designers: [
    {
      StageName: 'Blaze'
    },
    {
      StageName: 'Echo'
    },
    {
      StageName: 'Wilson'
    }
  ]
}
{
  _id: 'Red',
  designers: [
    {
      StageName: 'Gio'
    }
  ]
}
{

```

Figure 3.13: Query 1 Mongoshell output

This query uses the *aggregate()* function to process documents in the *Designer* collection. It begins by deconstructing the *Dress* array using the *\$unwind* operator, generating a separate document for each entry in the array. This step normalizes the data, allowing each dress to be treated as an individual document.

The resulting documents are then filtered using the *\$match* operator. Two conditions are applied: the first checks that the *Production_Time* of the dress satisfies the constraint imposed by the *\$lte* operator (less than or equal to a specified value), and the second filters the documents based on the color of the dress.

Finally, the query groups the documents by the dress color and uses the *\$addToSet* operator to create a list of unique *Stage_Names* of designers who have created dresses of each color. This approach ensures that each designer is listed only once per color, providing a

concise and organized summary of designer contributions grouped by dress color.

1 query with two conditions (evaluated separately) on a sub-document of an array

Query 1

Find model that have participated in exhibitions lasting less than 100 minutes and described as either “Pre-season Friendly” or “Spring Training Game”.

```
db.Model.find({
  "$and": [
    { "Model_Exhibition.Duration": { "$lt": 40 } },
    { "Model_Exhibition.Description": { "$in": ["Pre-season
      Friendly", "Spring Training Game"]} } ] },
  {"Name":1,"Surname":1,"Model_Exhibition":1}
})
```

```

> db.Model.find({
  "$and": [
    { "Model_Exhibition.Duration": { "$lt": 40 } },
    { "Model_Exhibition.Description": { "$in": ["Pre-season Friendly", "Spring Training Game"] } } },
  {"Name":1,"Surname":1,"Model_Exhibition":1}
)
< {
  _id: ObjectId('6749c93b0e659675aa2abdab'),
  Name: 'Sibby',
  Surname: 'Essery',
  Model_Exhibition: [
    {
      Exhibition_ID: 9,
      Address: '1 Randy Road',
      Description: 'Practice Scrimmage',
      Duration: 30,
      ID: 9,
      Title: 'Rebel Chic'
    },
    {
      Exhibition_ID: 11,
      Address: '99 Main Street',
      Description: 'Spring Training Game',
      Duration: 180,
      ID: 11,
      Title: 'Rebel Chic'
    },
    {
      Exhibition_ID: 15,
      Address: '56613 Londonderry Parkway',
      Description: 'Spring Training Game',
      Duration: 60,
      ID: 15,
      Title: 'Joy Bloom'
    },
    {
      Exhibition_ID: 17,
      Address: '1 Carpenter Terrace',
      Description: 'Spring Training Game',
      Duration: 60,
      ID: 17,
      Title: 'Echoes of Couture'
    }
  ]
}
{

```

Figure 3.14: Query 1 Mongoshell output

This query selects documents from the *Model* collection that meet multiple conditions on the *Model_Exhibition* field. The *\$and* operator is used to ensure that all specified

conditions are satisfied simultaneously.

The first condition filters exhibitions with a duration of less than 40 minutes, while the second condition checks that the *Description* of the exhibition is either *"Pre-season Friendly"* or *"Spring Training Game"*. These conditions allow for precise filtering of relevant exhibitions.

Projections are then applied to specify the fields to be returned in the query results. The selected fields include the *Name* and *Surname* of the models, along with the details of their *Model_Exhibition* entries.

1 query with two conditions (evaluated simultaneously) on a sub-document of an array

Query 1

Find models wearing dresses colored either Red, Turquoise or Orange that are made with 3 or more Fabric_Amount.

```
db.Model.find({"Dress": {"$elemMatch": {"Color": {"$in": ["Red", "Turquoise", "Orange"]}, "Fabric_Amount": {"$gte": 3}}}})
```

```

> db.Model.find(
  { "Dress": { "$elemMatch": { "Color": { "$in": ["Red", "Turquoise", "Orange"] },
    "Fabric_Amount": { "$gte": 3 } } } },
    {"Name":1,"Surname":1,"Model_Exhibition":1}
  )
< {
  _id: ObjectId('6749c93b0e659675aa2abdaa'),
  Name: 'Erick',
  Surname: 'Tapp',
  Model_Exhibition: [
    {
      Exhibition_ID: 5,
      Address: '906 Monument Trail',
      Description: 'Practice Scrimmage',
      Duration: 180,
      ID: 5,
      Title: 'Rebel Chic'
    },
    {
      Exhibition_ID: 6,
      Address: '02 Longview Road',
      Description: 'Spring Training Game',
      Duration: 45,
      ID: 6,
      Title: 'Naive Chic'
    },
    {
      Exhibition_ID: 10,
      Address: '4 Nelson Place',
      Description: 'Pre-season Friendly',
      Duration: 120,
      ID: 10,
      Title: 'Mystic Edge'
    }
  ]
}
{

```

Figure 3.15: Query 1 Mongoshell output

This query retrieves documents from the *Model* collection by applying filters to the *Dress* array. The *\$elemMatch* operator ensures that at least one element in the *Dress* array satisfies all specified conditions simultaneously.

The conditions include a filter on the *Color* field, where the *\$in* operator is used to specify a range of acceptable values, and a filter on the *Fabric_Amount* field, which must

be greater than or equal to three, as indicated by the *\$gte* operator.

This approach allows the query to precisely target documents where at least one dress in the array meets the defined criteria, making it effective for handling nested arrays with complex conditions.

List of Figures

2.1	MongoDB Collections Structure	3
3.1	Query 1 Mongoshell Output	6
3.2	Query 2 Mongoshell Output	7
3.3	Query 3 Mongoshell output	8
3.4	Query 4 Mongoshell output	9
3.5	Query 5 Mongoshell output	10
3.6	Query 1 Mongoshell output	11
3.7	Query 2 Mongoshell output	13
3.8	Query 1 Mongoshell output	15
3.9	Query 2 Mongoshell output	16
3.10	Query 1 Mongoshell output	17
3.11	Query 1 Mongoshell output	18
3.12	Query 1 Mongoshell output	20
3.13	Query 1 Mongoshell output	22
3.14	Query 1 Mongoshell output	24
3.15	Query 1 Mongoshell output	26

